

BALONCORE Roadmap

Purpose

BALONCORE is an AI-native security validation platform designed to help authorized teams find, prove, explain, and defend against real vulnerabilities across web, API, cloud, infrastructure, and Web3 systems.

The long-term goal is to build toward an XBOW-like class of product:

- autonomous or semi-autonomous security research
- agent-driven reconnaissance and hypothesis generation
- deterministic validation before reporting
- evidence-backed findings
- defensive guidance after proof
- company-grade SaaS and enterprise workflows

The operating principle is:

```
AI proposes.  
Validators prove or reject.  
Evidence becomes the product.  
Defense closes the loop.
```

BALONCORE must never become a tool that simply says "this might be vulnerable." The product must aim to prove what is real, reject weak hypotheses, and produce clear evidence that engineers, founders, auditors, and defenders can trust.

Product North Star

BALONCORE should become a security validation platform that can answer:

```
What can an attacker actually do?  
How do we know?  
What is the business impact?  
How do we fix it?  
How do we prevent it from coming back?
```

The platform should eventually support:

- Web and API pentesting
- SaaS authorization and tenant isolation testing
- Cloud and IAM attack-path validation
- Infrastructure and configuration review
- Web3 smart contract and protocol analysis
- CI/CD security gates
- AI-assisted reporting
- Defensive automation
- Enterprise evidence, audit, and compliance workflows

Safety Boundary

BALONCORE is for authorized security work only.

Allowed targets include:

- systems owned by the user or customer
- client-authorized environments
- explicit bug bounty scopes
- open-source repositories
- local labs and intentionally vulnerable apps
- CTFs and training environments

Core safety principles:

- scope must be configured before active testing
- deny rules override allow rules
- findings require evidence
- destructive actions require explicit future policy support
- validators must stay inside authorized scope
- reports must distinguish verified findings from hypotheses

Current Status

Current stage: **Stage 10 - Local Company-Grade Platform Pass (IMPLEMENTED, HARDENING CONTINUES)**

The repository now contains a working local/backend pass across Stages 2-10. This does not mean BALONCORE is a hosted enterprise SaaS or a fully autonomous internet-scale system yet; it means each roadmap stage now has executable code, artifacts, tests, and local smoke coverage that can be hardened into the next company/product version.

Completed Stage 2-10 product pass:

- ``index-evidence-run`` command for normalizing Stage 1 run artifacts into a durable local evidence store
- ``.baloncore/evidence/index.json`` store with scan, finding, hypothesis, report, policy, coverage, verification, and evidence bundle records
- per-scan evidence records under ``.baloncore/evidence/scans/``
- ``record-finding-lifecycle`` command for append-only finding lifecycle transitions
- lifecycle state enforcement for ``Hypothesis``, ``Rejected``, ``NeedsMoreEvidence``, ``Verified``, ``Reported``, ``Fixed``, ``Retested``, and ``Closed``
- finding lifecycle events embedded in both global and per-scan evidence records
- imported GLM lifecycle store primitives as a separate ``FindingStore`` for cross-domain findings, scan history, upsert, and state transitions
- imported CI baseline and policy gate primitives with severity-based blocking logic
- imported SARIF export primitives for GitHub/code-scanning style evidence output
- added ``export-evidence-sarif``, ``ci-evidence-gate``, and ``baseline-evidence`` commands on top of the Stage 2 evidence index
- imported defense recommendation engine with remediation, detection, regression-test, WAF, least-privilege, and threat-model guidance
- added ``defense-report`` command for classification-based defensive reporting
- imported cloud IAM graph and provider parsers for AWS, GCP, Azure, Kubernetes, Terraform state, and CloudFormation

- added `analyze-cloud-iam` command with offline artifacts and lifecycle finding-store export
- imported Web3/Solidity analysis, Solidity parser, Slither JSON ingestion, Foundry invariant generation, and fuzz-output parsing primitives
- added `analyze-web3` command with local Web3 artifacts and lifecycle finding-store export
- added local authorized labs for cloud IAM and Web3 analysis under `labs/cloud-iam/` and `labs/vulnerable-protocol/`
- Stage 2 customer-grade evidence report export for HTML, Markdown, and JSON
- Stage 2 export-blocking sensitive evidence check plus redaction command
- Stage 3 versioned agent prompt/contract registry and deterministic fixture agent runner
- Stage 3 verifier challenge loop and validator bridge that prevents model claims from becoming findings
- Stage 3 CLI commands: `show-agent`, `run-agent-dry-run`, `validate-agent-output`, and `agent-pipeline`
- Stage 4 self-contained local dashboard export with scan history, findings, evidence integrity, hypotheses, redaction blockers, and report links
- Stage 5 combined `ci-run` suite for evidence gating, SARIF, PR Markdown summaries, evidence bundle policy, and redaction checks
- Stage 5 OpenAPI diff command and Slack/Linear/Jira dry-run notification payload exporter
- Stage 6 cloud reachability proof artifacts and cloud defense plan output
- Stage 7 Web3 generated Foundry proof-test manifest and generated `foundry-tests/\*.t.sol` artifacts
- Stage 8 bounded autonomous orchestration loop with recon, hypothesis, validator bridge, reporting, budgets, noise policy, and approval gates
- Stage 9 defense bundle export with prevention, detection, regression-test snippets, and retest plans
- Stage 10 local platform model with organizations, workspaces, users, RBAC, billing, audit log, compliance mapping, onboarding, usage metrics, and access checks

GLM copy assessment:

- GLM reached broad **Stage 2 and early Stage 3/4/5 scaffolding**, plus some Stage 6, Stage 7, and Stage 8 sketches.
- The useful, test-passing backend pieces were ported selectively.
- The dashboard, API server, fixture-only agents, autonomous pentest scaffold, and tenant/SaaS layer were not ported because they were not mature enough for the main branch yet.

Completed foundation work:

- Rust workspace
- `baloncore-core` crate
- `baloncore` CLI crate
- config format
- scope guard
- evidence model
- finding model
- proof package model
- blast-radius safety policy
- schema discovery probe registry
- attack-chain rule registry

- built-in agent specs
- safety documentation
- architecture documentation
- SimdiaScanAI review and feature import notes
- initial unit tests

Completed Stage 1 work:

- endpoint inventory model
- HTTP method and endpoint source model
- HTTP exchange model
- two-user auth profile setup in `baloncore.toml`
- deterministic BOLA/IDOR validator
- synthetic BOLA proof demo CLI
- blast-radius estimate CLI
- triage gate primitives
- redaction-aware evidence metadata
- matcher/candidate primitives
- append-only pipeline event primitives
- local vulnerable API lab
- scoped live HTTP request runner
- live `validate-lab-idor` command
- local artifact writer for exchanges, decisions, proof packages, and Markdown report
- OpenAPI JSON endpoint inventory importer
- live multi-schema discovery runner for OpenAPI, GraphQL, Postman, WSDL, and gRPC reflection probes
- discovered-schema candidate generation for actively validatable non-OpenAPI endpoints
- safe collection endpoint seed candidate selector
- object seed extraction from owner-authenticated JSON responses
- `--seed-file` and manual `--object-id` seed support
- authenticated object-endpoint candidate selector
- `scan-openapi-bola` command for owner/profile/anonymous auth-profile matrix validation
- multi-profile matrix defaults across all non-owner configured profiles
- explicit anonymous validation records for missing-authentication detection with impact/remediation artifacts
- CI policy gate `scan-openapi-bola --ci-anonymous-exposure` for protected endpoint anonymous exposure
- configurable active-check noise mode (`quiet`, `moderate`, `loud`) with per-validation noise tagging
- hard active-request budget enforcement with `--max-active-requests`
- authorization classification for BOLA, BFLA, missing authentication, intended privileged access, intended owner access, and blocked-as-expected cases
- resource-aware seed/candidate matching to avoid unrelated endpoint/object probes
- explicit `resource\_mismatch` skip records in matrix summaries

- response-shape comparison for owner/tested responses
- sensitive-field detection for email, financial, ownership, credential, and privileged business data
- impact severity scoring and ``impact.json`` artifacts
- executive ``run_report.md`` aggregation with severity-ranked findings and coverage/noise-control counts
- OpenAPI coverage artifacts with tested, skipped, seed-only, and not-eligible endpoint status
- ``openapi_coverage.json`` and ``coverage_report.md``
- workflow family and state-transition mapping artifacts from discovered endpoints
- explicit hypothesis lifecycle ledger (``hypothesis_ledger.json``) with proposed/promoted/rejected/suppressed traces
- persistent hypothesis memory in ``.baloncore/hypotheses/index.json``
- config-driven suppression rules for approved access patterns
- suppressed-finding accounting plus ``suppression.json`` evidence artifacts
- one-command Stage 1 E2E runner: ``scripts/stage1_e2e_demo.sh``
- Stage 1 E2E runner defaults to dedicated local demo port ``31337``
- Stage 1 E2E runner supports ``--json`` summaries and ``--fail-on-regression`` policy gating
- persistent local finding memory in ``.baloncore/findings/index.json``
- stable finding fingerprints with new/recurring status in run reports
- remediation plans for verified authorization findings
- replayable regression checks for owner, tested-profile, and anonymous access
- ``remediation.json`` and ``remediation.md`` artifacts in proof folders
- executable ``run-regression`` command for remediation plans
- ``regression_result.json`` and ``regression_result.md`` fixed-state verdict artifacts
- ``run-regression --ci`` pipeline mode that fails builds on ``StillFailing``
- ``export-regression-ci`` GitHub Actions workflow generation
- tamper-evident ``evidence_manifest.json`` bundles with SHA-256 file hashes
- ``seal-evidence`` and ``verify-evidence`` commands for evidence integrity checks
- local Ed25519 signing key generation with ``init-signing-key``
- ``sign-evidence`` command that writes ``evidence_signature.json``
- signature-aware ``verify-evidence --require-signature``
- signer trust registry with ``trust-evidence-signer``
- trusted-only evidence verification with ``verify-evidence --trusted-only``
- batch run verification with ``verify-evidence-run``
- CI evidence gate with ``verify-evidence-run --trusted-only --ci``
- tamper-negative tests for modified evidence files and modified signatures
- per-candidate/per-seed proof folders, ``object_seeds.json``, and ``matrix_summary.json``
- BOLA validator tests for verified, blocked, public-object, and unrelated-response cases
- OpenAPI inventory unit test
- object seed extraction unit test
- authorization matrix classification tests for BOLA, BFLA, and intended admin access

- resource matching tests for matched, mismatched, and manual seed cases
- response impact tests for sensitive BOLA and privileged BFLA findings
- remediation-plan unit test for verified BOLA findings
- regression verdict unit test for fixed and still-failing patch states

Stage 1 exit criteria status:

- local lab API scanning: complete
- intentionally planted IDOR/BOLA proof: complete
- false-positive and resource-mismatch rejection: complete
- evidence storage for verified results: complete
- signed evidence and CI gates: complete
- schema discovery, workflow mapping, noise policy, and hypothesis lifecycle hardening: complete enough to graduate to Stage 2

Working commands:

```
cargo run -p baloncore -- init
cargo run -p baloncore -- check-config baloncore.toml
cargo run -p baloncore -- check-scope http://localhost:3000/api/users
cargo run -p baloncore -- explain-agents
cargo run -p baloncore -- estimate-blast-radius --endpoints 50 --params 4 --variants 10 --depth medium --auth
cargo run -p baloncore -- demo-idor
cargo run -p baloncore -- validate-lab-idor --base-url http://127.0.0.1:3000
cargo run -p baloncore -- scan-openapi-bola
cargo run -p baloncore -- run-regression .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/remediation.j
cargo run -p baloncore -- run-regression .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/remediation.j
cargo run -p baloncore -- export-regression-ci .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/remedia
cargo run -p baloncore -- seal-evidence .baloncore/runs/<run-id>/candidate-002-seed-001/user_a
cargo run -p baloncore -- init-signing-key
cargo run -p baloncore -- sign-evidence .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/evidence_manif
cargo run -p baloncore -- trust-evidence-signer .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/eviden
cargo run -p baloncore -- verify-evidence .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/evidence_man
cargo run -p baloncore -- verify-evidence .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/evidence_man
cargo run -p baloncore -- verify-evidence .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/evidence_man
cargo run -p baloncore -- verify-evidence-run .baloncore/runs/<run-id> --trusted-only --ci
cargo run -p baloncore -- index-evidence-run .baloncore/runs/<run-id>
cargo run -p baloncore -- record-finding-lifecycle <scan-id> <finding-id> --state Reported --reason "Included
cargo run -p baloncore -- export-evidence-sarif --output .baloncore/evidence/report.sarif
cargo run -p baloncore -- ci-evidence-gate --ci
cargo run -p baloncore -- baseline-evidence
cargo run -p baloncore -- defense-report BrokenObjectLevelAuthorization --output .baloncore/defense/bola.md
cargo run -p baloncore -- analyze-cloud-iam aws labs/cloud-iam/aws-risky.json
cargo run -p baloncore -- analyze-web3 labs/vulnerable-protocol
cargo run -p baloncore -- export-evidence-report --format html --output .baloncore/evidence/evidence_report.h
cargo run -p baloncore -- check-evidence-export .baloncore/runs/<run-id>
cargo run -p baloncore -- redact-evidence-file .baloncore/runs/<run-id>/candidate-002-seed-001/user_a/tested_
cargo run -p baloncore -- show-agent api-auth --json
cargo run -p baloncore -- agent-pipeline api-auth --scan-dir .baloncore/runs/<run-id>
cargo run -p baloncore -- validate-agent-output api-auth .baloncore/agents/<artifact>.json
cargo run -p baloncore -- export-dashboard --store .baloncore/evidence/index.json
cargo run -p baloncore -- ci-run --check-redaction --sarif-output .baloncore/ci/baloncore.sarif
cargo run -p baloncore -- diff-openapi current-openapi.json previous-openapi.json --json
cargo run -p baloncore -- export-notification-dry-run .baloncore/ci/pr_summary.md --adapter slack
cargo run -p baloncore -- autonomous-run --scan-dir .baloncore/runs/<run-id>
cargo run -p baloncore -- export-defense-bundle --evidence-store .baloncore/evidence/index.json
cargo run -p baloncore -- platform-bootstrap
cargo run -p baloncore -- platform-check-access --user user-owner-example-test --workspace ws-production --pe
```

Verified:

```
cargo fmt --check
```

cargo test

SimdiaScanAI Feature Import

Before continuing into Stage 1, the legacy `SimdiaScanAI` folder was reviewed for ideas worth preserving.

Useful concepts imported into BALONCORE:

- proof packages for reproduction-ready evidence
- blast-radius estimation before active testing
- schema discovery for OpenAPI, GraphQL, Postman, WSDL, and gRPC
- attack-chain reasoning from verified atomic findings
- business-logic mapping for user flows and state transitions
- methodology/checklist thinking for later guided scan modes

Ideas deliberately not imported directly:

- generated certificates and keys
- `.venv`, `node\_modules`, `dist`, and cache folders
- old Python service stack
- stealth or WAF-bypass modules
- monetization and negotiation automation
- large frontend implementation

Decision:

Keep the best concepts.
Do not inherit the old architecture.
BALONCORE is the cleaner Rust-first successor.

Detailed review:

docs/SIMDIASCANAI_REVIEW.md

External Repo Review

The following public security projects were reviewed before continuing Stage 1:

- ``https://xbow.com/``
- ``https://github.com/xbow-security``
- ``https://github.com/elementalsouls/Claude-BugHunter``
- ``https://github.com/shuvonsec/claude-bug-bounty``
- ``https://github.com/0xSteph/pentest-ai-agents``
- ``https://github.com/nullthrix/BugBounty-Methodology``
- ``https://github.com/zakirkun/deep-eye``
- ``https://github.com/vercel-labs/deepsec``
- ``https://github.com/xdavidhu/awesome-google-vrp-writeups``

Imported BALONCORE-native ideas:

- triage gate before validation/reporting

- evidence hygiene and redaction states
- matcher/candidate model before expensive AI processing
- append-only scan pipeline and run history
- detection engineering output after verified findings
- noise-level and OPSEC tracking for active commands
- autonomous loop memory and recon ranking
- writeup-derived attack-pattern corpus

No third-party source code was copied into BALONCORE. The import was conceptual and architectural.

Detailed review:

`docs/EXTERNAL_REPO_REVIEW.md`

Architecture Vision

Rust Kernel

Rust should own the reliable core:

- target model
- scope enforcement
- evidence model
- finding lifecycle
- validator orchestration
- attack graph primitives
- CLI workflows
- local scan runner
- high-performance parsing where needed

TypeScript Layer

TypeScript should own product and browser-heavy work:

- dashboard
- user interface
- Playwright browser automation
- GitHub, Slack, Linear, Jira integrations
- SaaS API layer if needed

Python Layer

Python can support research and tool glue:

- AI orchestration experiments
- wrappers around existing security tools
- report experiments
- vulnerability lab utilities

- ML or heuristic prototypes

Agent Layer

Agents should be versioned modules with strict contracts:

- mission
- inputs
- forbidden actions
- output schema
- confidence rules
- validation requirement

Agents should not directly create final findings unless the finding is backed by validator evidence.

Legacy-Derived Product Principles

The SimdiaScanAI review reinforced five product principles:

- proof packages should be generated for every verified finding
- scan impact must be estimated before active testing
- API schemas should be discovered before fuzzing or authorization checks
- business workflows are first-class attack surface
- attack chains must be built from verified atomic findings, not model guesses

Stage 0 - Foundation

Goal

Create the serious base for the product so future modules can plug into a consistent kernel.

Core Capabilities

- repository structure
- Rust workspace
- config format
- scope model
- safety policy
- evidence model
- finding model
- agent specs
- basic CLI
- tests

Deliverables

- ``README.md``
- ``docs/BLUEPRINT.md``
- ``docs/ARCHITECTURE.md``
- ``docs/SAFETY.md``
- ``Baloncore_roadmap.md``
- ``baloncore.toml``
- ``crates/baloncore-core``
- ``crates/baloncore-cli``
- ``agents/recon.md``
- ``agents/api-auth.md``
- ``agents/verifier.md``
- ``agents/reporter.md``

Exit Criteria

- project builds
- config can be generated and validated
- scope decisions can be checked
- agent specs can be listed
- tests pass

Status

Complete enough to support Stage 1 development. The kernel remains intentionally small, but it now includes the first Web/API validation primitives.

Stage 1 - Web/API Validation MVP

Goal

Build the first useful security engine: web and API authorization validation for authorized targets.

This is the first product wedge because it is valuable, understandable, demo-friendly, and relevant to startups, SaaS companies, and internal engineering teams.

Target Vulnerability Classes

- broken object-level authorization
- IDOR
- broken function-level authorization
- missing authentication
- tenant isolation failure
- role confusion
- unsafe direct object references
- schema and implementation mismatch
- sensitive endpoint exposure

Core Features

- target URL intake
- scope enforcement on every request
- triage gate before expensive validation
- endpoint inventory
- schema discovery for OpenAPI, GraphQL, Postman, WSDL, and gRPC
- matcher/candidate discovery for source and endpoint signals
- HTTP request and response capture
- auth profile model
- anonymous, user A, user B, admin profiles
- object identifier extraction
- replay request under alternate users
- compare response status, body shape, and sensitive fields
- business-logic flow inventory
- state-transition and workflow-order mapping
- blast-radius estimate before active validation
- quiet/moderate/loud noise tagging for active checks
- generate hypothesis records
- promote only reproducible findings

First "Wow" Feature

BALONCORE should create or use multiple authorized test users, map their accessible objects, replay object requests across roles, and prove when user A can access user B's data.

Example proof:

```
User A requested /api/invoices/inv_2002.  
The invoice belongs to User B.  
The server returned HTTP 200 with User B's invoice details.  
BALONCORE captured both the baseline and attack request.
```

Deliverables

- endpoint model - started
- auth profile model
- schema discovery model
- business workflow model
- blast-radius gate - started
- triage gate - started
- matcher/candidate model - started
- HTTP exchange evidence type - started
- request runner - started
- response comparator
- BOLA validator - started
- missing-auth validator
- initial JSON or Markdown report

Exit Criteria

- BALONCORE can scan a local lab API
- it can prove at least one intentionally planted IDOR
- it can reject at least one false positive
- it stores evidence for each verified result

Stage 2 - Evidence And Reporting Engine

Goal

Turn raw validation into reports that customers, founders, engineers, auditors, and investors can trust.

Core Features

- evidence database
- append-only run history
- proof package generator
- proof package JSON output - started
- scan IDs
- finding IDs
- finding lifecycle
- hypothesis status
- rejected hypothesis records
- verified finding records
- HTTP exchange viewer data
- reproduction-ready curl command
- optional screenshot, video, timing, and OOB callback evidence
- evidence sensitivity labels
- redaction status and export blocking when redaction is required
- severity scoring
- Markdown report export
- first Markdown report output - started
- HTML report export
- executive summary
- engineering fix guidance
- regression test suggestions

Finding Lifecycle

Hypothesis
Rejected
Needs More Evidence
Verified
Reported
Fixed
Retested
Closed

Deliverables

- evidence storage
- proof package storage
- report renderer

- severity model
- finding state machine
- sample reports
- report templates

Exit Criteria

- every finding links to evidence
- unverified hypotheses are excluded from final reports
- reports can be shared with a technical team
- reports include reproduction, impact, and fix guidance

Stage 3 - AI Agent Layer

Goal

Add intelligence while preventing hallucinated findings.

Agent Types

- Recon Agent
- Schema Discovery Agent
- API Auth Agent
- Business Logic Agent
- Attack Chain Agent
- Triage Agent
- Evidence Hygiene Agent
- Matcher Author Agent
- Verifier Agent
- Revalidation Agent
- Detection Engineer Agent
- Reporter Agent
- Fix Guidance Agent
- Defense Agent

Core Features

- versioned prompts
- strict output schemas
- model-independent agent contracts
- shared scope guard for execution-capable agents
- hypothesis generation
- schema-aware endpoint reasoning
- business-flow reasoning
- attack-chain hypothesis generation from verified findings
- reasoning over endpoint inventory
- finding explanation
- false-positive challenge
- evidence summarization
- refusal and skipped-work tracking

Critical Rule

AI can propose a vulnerability path, but only validators can promote it into a verified finding.

Deliverables

- agent runner
- structured agent I/O
- prompt registry
- JSON schema validation
- model configuration
- verifier challenge loop

Exit Criteria

- AI can generate useful hypotheses from endpoint data
- AI can use schema discovery results to prioritize tests
- AI can propose business-logic validation plans without executing unsafe actions
- weak hypotheses are rejected
- final reports cite evidence instead of model claims
- agent outputs are repeatable enough for product use

Stage 4 - Dashboard

Goal

Make BALONCORE usable, demoable, and product-grade.

Core Views

- scan creation
- scope editor
- auth profile manager
- endpoint inventory
- attack surface map
- findings list
- evidence viewer
- report preview
- scan history
- system settings

Design Direction

BALONCORE should feel like a serious security operations product:

- dense but clear
- evidence-first
- fast to scan
- restrained visual style
- strong information hierarchy
- no unnecessary marketing surface inside the app

Deliverables

- web dashboard
- API service
- scan detail page
- evidence viewer
- finding detail page
- report export controls

Exit Criteria

- a user can run and inspect a scan without the CLI
- evidence is easy to review
- findings are clearly separated by status
- the product can be demoed to a founder or investor

Stage 5 - CI/CD And GitHub Integration

Goal

Make BALONCORE useful before code reaches production.

Core Features

- GitHub repository connection
- pull request scans
- GitHub Actions integration
- SARIF output
- diff-only candidate discovery
- append-only file and endpoint records
- revalidation of existing findings after code changes
- release-blocking policies
- Slack notifications
- Linear or Jira ticket creation
- baseline comparisons
- retest fixed findings

Use Cases

- block a release when a verified authorization bug is introduced
- comment on pull requests with evidence
- generate regression tests for confirmed bugs
- track fixed versus unresolved issues

Deliverables

- GitHub integration
- CI command mode
- SARIF exporter
- policy config
- notification adapters

Exit Criteria

- BALONCORE can run in CI
- results can appear in GitHub security tooling
- teams can distinguish new findings from existing findings
- verified critical issues can fail a build

Stage 6 - Cloud/IAM Attack Path Module

Goal

Expand into cloud and infrastructure defense by proving risky cloud relationships and privilege paths.

Target Areas

- AWS IAM
- GCP IAM
- Azure IAM
- Terraform
- CloudFormation
- Kubernetes
- GitHub Actions to cloud trust
- public storage exposure
- dangerous role assumptions
- over-permissive service accounts

Core Features

- cloud config ingestion
- IaC scanning
- IAM graph model
- principal-to-permission path analysis
- public exposure checks
- CI/CD identity checks
- cloud evidence records
- remediation guidance

Example "Wow" Finding

This GitHub Actions workflow can request an OIDC token.
The token can assume AWS role DeployRole.
DeployRole can pass AdminRole to a compute service.
That path gives effective administrative control.

Deliverables

- IAM graph engine
- Terraform adapter
- Checkov or similar integration
- Prowler or similar integration
- attack-path report format

Exit Criteria

- BALONCORE can explain at least one real privilege path

- it can distinguish theoretical permission from reachable permission
- cloud findings include exact identity chain evidence

Stage 7 - Web3 Smart Contract Module

Goal

Expand into crypto protocol security with invariant generation and exploit validation.

Target Areas

- Solidity
- Foundry
- Hardhat
- DeFi accounting
- oracle usage
- liquidation logic
- bridge/message-passing logic
- signature replay
- upgradeability
- access control
- reentrancy
- rounding and decimal mismatches

Core Features

- Foundry and Hardhat ingestion
- Slither integration
- invariant generation
- fuzzing orchestration
- property-based tests
- symbolic or heuristic path discovery
- exploit proof tests
- protocol-specific report templates

Example "Wow" Finding

BALONCORE generated an invariant for total share accounting.
The fuzzer found a sequence where withdraw and donate operations desynchronize assets and shares.
BALONCORE produced a Foundry test that reproduces the issue.

Deliverables

- Web3 project adapter
- Slither adapter
- Foundry test generator
- invariant template library
- fuzz runner
- Web3 finding model extensions

Exit Criteria

- BALONCORE can run against a local vulnerable protocol
- it can generate at least one useful invariant
- it can produce a reproducible Foundry test for a confirmed issue

Stage 8 - Autonomous Multi-Agent Pentest Mode

Goal

Move toward the XBOW-like experience: scoped autonomous research loops with validation and reporting.

Agent Roles

- Planner Agent
- Recon Agent
- Tool Agent
- Web/API Agent
- Cloud Agent
- Web3 Agent
- Exploit Validation Agent
- Verifier Agent
- Reporter Agent
- Defense Agent

Core Features

- task planning
- tool selection
- scan memory
- candidate memory
- run history
- attack graph memory
- verified atomic finding graph
- chain-edge confidence and assumptions
- automatic retesting
- hypothesis prioritization
- recon ranking
- false-positive courtroom
- bounded autonomous execution
- human approval gates for risky actions

Autonomous Loop

Plan
Recon
Hypothesize
Validate
Reject or Verify
Collect Evidence
Report
Defend
Retest

Deliverables

- agent orchestrator
- task queue
- memory store
- attack graph
- validator registry
- human approval controls

Exit Criteria

- BALONCORE can run a scoped pentest workflow
- it can choose between multiple validators
- it can explain why a path was abandoned
- it can produce verified findings without manual report writing

Stage 9 - Defensive Automation

Goal

Make BALONCORE a defense platform, not only an offensive testing tool.

Core Features

- fix guidance
- secure code patch suggestions
- regression test generation
- WAF rule suggestions
- SIEM query suggestions
- Sigma-style rule output
- cloud audit-log detection guidance
- logging recommendations
- IAM least-privilege suggestions
- detection logic
- threat model updates
- retest workflows

Defensive Output Examples

- unit test or integration test for the bug
- route-level authorization check recommendation
- tenant isolation regression test
- IAM policy reduction
- cloud alert rule
- log event to monitor future abuse

Deliverables

- defense recommendation engine
- regression test generator
- detection rule templates
- remediation tracking
- retest scheduler

Exit Criteria

- every verified finding includes prevention guidance
- every major finding includes detection guidance where applicable
- fixed findings can be retested
- teams can prove that a bug did not return

Stage 10 - Company-Grade Platform

Goal

Turn BALONCORE into a fundable and sellable product.

Core Features

- multi-tenant SaaS
- user accounts
- organizations and teams
- RBAC
- enterprise SSO
- encrypted evidence storage
- audit logs
- billing
- usage limits
- onboarding flows
- deployment options
- customer workspaces
- compliance mapping
- support and admin tools

Commercial Positioning

BALONCORE should be positioned as:

AI security validation that proves real risk and helps teams defend.

Target customers:

- startups shipping APIs quickly
- SaaS companies with tenant isolation risk
- Web3 protocols and audit teams
- cloud-heavy engineering teams
- security teams that need validated findings, not scanner noise
- founders who need pre-production security assurance

Investor-Grade Metrics

- verified findings per scan
- false-positive reduction rate
- time to proof
- time to fix
- retest success rate
- number of CI-blocked critical issues

- customer adoption
- scan volume
- recurring usage
- expansion across modules

Exit Criteria

- customers can onboard without manual setup
- teams can run repeatable scans
- reports are trusted by engineering teams
- product has a clear paid use case
- the platform can support multiple customers securely

Release Milestones

v0.1 - Kernel

- CLI
- config
- scope guard
- evidence model
- finding model
- agent specs

v0.2 - Local Web/API Validator

- endpoint inventory
- matcher/candidate model
- auth profiles
- HTTP exchange capture
- triage gate
- blast-radius estimate
- BOLA/IDOR checks
- local report output

v0.3 - Evidence Reports

- finding lifecycle
- append-only run history
- evidence database
- evidence redaction/hygiene
- Markdown and HTML reports
- fix guidance

v0.4 - AI Hypothesis Engine

- structured agent runner
- recon and API auth agents
- schema-discovery, business-logic, triage, and evidence-hygiene agents
- verifier challenge loop
- schema-constrained outputs

v0.5 - Dashboard

- scan UI
- evidence viewer
- findings UI
- reports UI

v0.6 - CI/GitHub

- GitHub Actions mode
- SARIF export
- pull request comments
- baseline comparisons

v0.7 - Cloud/IAM

- IaC ingestion
- IAM graph
- cloud attack paths

v0.8 - Web3

- Foundry/Hardhat ingestion
- Slither integration
- invariant generation
- fuzzing workflow

v0.9 - Autonomous Pentest Mode

- multi-agent planner
- memory
- attack graph reasoning
- validator registry

v1.0 - Company Platform

- SaaS dashboard
- teams
- RBAC

- encrypted evidence storage
- billing-ready product shape

What We Build Next

The current work is **Stage 1 - Web/API Validation MVP**.

Immediate implementation plan:

1. Add endpoint inventory types. Done. 2. Add HTTP request and response evidence records. Started. 3. Add auth profile details for test users. Done. 4. Add a blast-radius gate for active validation. Started. 5. Implement a first BOLA/IDOR validator. Started. 6. Add triage gate and redaction-aware evidence. Done. 7. Add matcher/candidate and append-only pipeline primitives. Done. 8. Build a local vulnerable lab API. Done. 9. Build a scoped request runner. Started. 10. Add schema discovery execution for OpenAPI, GraphQL, Postman, WSDL, and gRPC sources. 11. Add proof package records to verified findings. Started. 12. Store verified and rejected results. Started. 13. Generate a first Markdown report. Started.

The first concrete success target:

BALONCORE scans a local lab API and proves one intentionally planted IDOR with request and response evidence.

That is the point where BALONCORE stops being only architecture and starts becoming a security product.